Ingeniería del Software I

Nazareno Aguirre, Laura Brandán, Franco Luque, Matías Bordese, Silvia Pelozo

Ingeniería del Software I

Nazareno Aguirre, Laura Brandán, Franco Luque, Matías Bordese, Silvia Pelozo

Revisión de Orientación a Objetos

Abstracción y Descomposición

Abstracción y Descomposición

Uno de los problemas fundamentales a resolver cuando se construye un sistema de software es cómo lidiar con la complejidad del mismo. La estrategia más natural para lograr resolver problemas de gran magnitud es "dividir para vencer".

Abstracción y Descomposición

Uno de los problemas fundamentales a resolver cuando se construye un sistema de software es cómo lidiar con la complejidad del mismo. La estrategia más natural para lograr resolver problemas de gran magnitud es "dividir para vencer".

Dividir para vencer corresponde a la idea de descomponer el problema en subproblemas, de manera tal que, al resolver los subproblemas, se consiga resolver el problema original.

Abstracción y Descomposición

Uno de los problemas fundamentales a resolver cuando se construye un sistema de software es cómo lidiar con la complejidad del mismo. La estrategia más natural para lograr resolver problemas de gran magnitud es "dividir para vencer".

Dividir para vencer corresponde a la idea de descomponer el problema en subproblemas, de manera tal que, al resolver los subproblemas, se consiga resolver el problema original.

Esta técnica es efectiva sólo si la descomposición de problemas en subproblemas es adecuada. Esto en general significa que

Abstracción y Descomposición

Uno de los problemas fundamentales a resolver cuando se construye un sistema de software es cómo lidiar con la complejidad del mismo. La estrategia más natural para lograr resolver problemas de gran magnitud es "dividir para vencer".

Dividir para vencer corresponde a la idea de descomponer el problema en subproblemas, de manera tal que, al resolver los subproblemas, se consiga resolver el problema original.

Esta técnica es efectiva sólo si la descomposición de problemas en subproblemas es adecuada. Esto en general significa que

- los subproblemas deben ser relativamente independientes (deben poder resolverse independientemente unos de otros), y

Abstracción y Descomposición

Uno de los problemas fundamentales a resolver cuando se construye un sistema de software es cómo lidiar con la complejidad del mismo. La estrategia más natural para lograr resolver problemas de gran magnitud es "dividir para vencer".

Dividir para vencer corresponde a la idea de descomponer el problema en subproblemas, de manera tal que, al resolver los subproblemas, se consiga resolver el problema original.

Esta técnica es efectiva sólo si la descomposición de problemas en subproblemas es adecuada. Esto en general significa que

- los subproblemas deben ser relativamente independientes (deben poder resolverse independientemente unos de otros), y
- los subproblemas deben poder combinarse de manera de solucionar el problema original.

Abstracción y Descomposición

Abstracción y Descomposición

Otro elemento fundamental para lidiar con la complejidad en la programación es la abstracción. Esencialmente la abstracción consiste en la eliminación de detalles para simplificar problemas.

Abstracción y Descomposición

Otro elemento fundamental para lidiar con la complejidad en la programación es la abstracción. Esencialmente la abstracción consiste en la eliminación de detalles para simplificar problemas.

Por supuesto, para que la abstracción sea útil la eliminación de detalles **no** puede ser arbitraria: deben eliminarse sólo aquellos detalles que no son relevantes para la solución del problema en cuestión.

Abstracción y Descomposición

Otro elemento fundamental para lidiar con la complejidad en la programación es la abstracción. Esencialmente la abstracción consiste en la eliminación de detalles para simplificar problemas.

Por supuesto, para que la abstracción sea útil la eliminación de detalles **no** puede ser arbitraria: deben eliminarse sólo aquellos detalles que no son relevantes para la solución del problema en cuestión.

Un ejemplo clásico de abstracción es la que se consigue mediante el uso de lenguajes de programación de alto nivel. Estos lenguajes "esconden" numerosos detalles al programador, tales como detalles de la planificación de procesos por parte del sistema operativo, detalles de representación de tipos de datos elementales, detalles sobre los dispositivos físicos específicos disponibles en la plataforma en que se corre el programa, etc.

Mecanismos de Estructuración de Programas

Mecanismos de Estructuración de Programas

Los lenguajes de programación ofrecen en general mecanismos para estructurar programas. Estos mecanismos nos permiten organizar los programas en partes, brindando construcciones para explotar la descomposición y la abstracción.

Mecanismos de Estructuración de Programas

Los lenguajes de programación ofrecen en general mecanismos para estructurar programas. Estos mecanismos nos permiten organizar los programas en partes, brindando construcciones para explotar la descomposición y la abstracción.

Revisaremos aquí algunos de los mecanismos de estructuración de programas más frecuentemente usados en la programación imperativa.

Mecanismos de Estructuración de Programas

Los lenguajes de programación ofrecen en general mecanismos para estructurar programas. Estos mecanismos nos permiten organizar los programas en partes, brindando construcciones para explotar la descomposición y la abstracción.

Revisaremos aquí algunos de los mecanismos de estructuración de programas más frecuentemente usados en la programación imperativa.

Dado que en la materia abarcaremos la programación orientada a objetos, esta revisión nos servirá además para justificar algunas de las ventajas de la programación orientada a objetos (sobre la programación imperativa), observando algunas limitaciones de los lenguajes imperativos no OO.

Mecanismos de Estructuración de Programas

Mecanismos de Estructuración de Programas

Sistemas Monolíticos

Mecanismos de Estructuración de Programas

Sistemas Monolíticos

La técnica de programación imperativa más simple consiste en programar utilizando un único programa, el programa principal, que modifica datos almacenados en variables globales.

Mecanismos de Estructuración de Programas

Sistemas Monolíticos

La técnica de programación imperativa más simple consiste en programar utilizando un único programa, el programa principal, que modifica datos almacenados en variables globales.

Esta técnica es sólo útil para programas muy cortos, de sólo unas cuantas líneas. Pero para programas de algunas decenas de líneas de código (o más grandes), la técnica es completamente inútil.

Mecanismos de Estructuración de Programas

Sistemas Monolíticos

La técnica de programación imperativa más simple consiste en programar utilizando un único programa, el programa principal, que modifica datos almacenados en variables globales.

Esta técnica es sólo útil para programas muy cortos, de sólo unas cuantas líneas. Pero para programas de algunas decenas de líneas de código (o más grandes), la técnica es completamente inútil.

Mecanismo de reutilización asociado a la técnica: "copy & paste".

Mecanismos de Estructuración de Programas

Sistemas Monolíticos

La técnica de programación imperativa más simple consiste en programar utilizando un único programa, el programa principal, que modifica datos almacenados en variables globales.

Esta técnica es sólo útil para programas muy cortos, de sólo unas cuantas líneas. Pero para programas de algunas decenas de líneas de código (o más grandes), la técnica es completamente inútil.

Mecanismo de reutilización asociado a la técnica: "copy & paste".

Problemas: Dificultad de comprensión del programa, dificulta el reuso, la depuración, y muchos otros problemas (demasiados para enumerar aquí).

Mecanismos de Estructuración de Programas

Mecanismos de Estructuración de Programas

Sistemas Monolíticos

Mecanismos de Estructuración de Programas

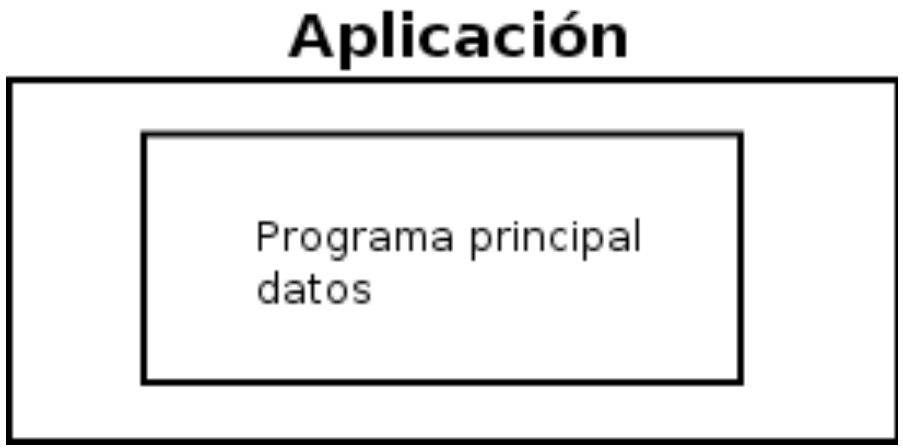
Sistemas Monolíticos

Gráficamente, un sistema monolítico tiene la siguiente estructura:

Mecanismos de Estructuración de Programas

Sistemas Monolíticos

Gráficamente, un sistema monolítico tiene la siguiente estructura:



Mecanismos de Estructuración de Programas

Programación con Procedimientos

Mecanismos de Estructuración de Programas

Programación con Procedimientos

Basándose en los problemas de la programación sin modularización, surge la idea de separar las porciones de código que se necesitan reutilizar en procedimientos.

Mecanismos de Estructuración de Programas

Programación con Procedimientos

Basándose en los problemas de la programación sin modularización, surge la idea de separar las porciones de código que se necesitan reutilizar en procedimientos.

Básicamente, el programa principal se descompone en diferentes procedimientos. Éstos pueden ser invocados, lo cual provoca la ejecución del cuerpo del procedimiento, y el posterior retorno a la "posición siguiente" a la correspondiente a la invocación.

Mecanismos de Estructuración de Programas

Programación con Procedimientos

Basándose en los problemas de la programación sin modularización, surge la idea de separar las porciones de código que se necesitan reutilizar en procedimientos.

Básicamente, el programa principal se descompone en diferentes procedimientos. Éstos pueden ser invocados, lo cual provoca la ejecución del cuerpo del procedimiento, y el posterior retorno a la "posición siguiente" a la correspondiente a la invocación.

Gráficamente, la situación es la siguiente:

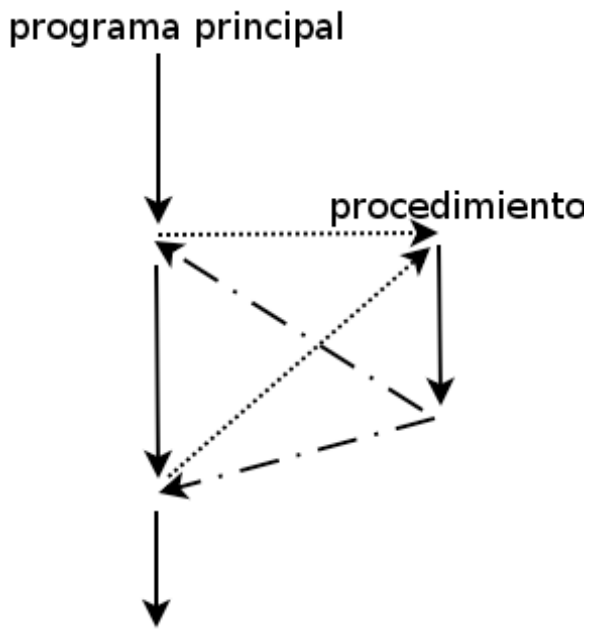
Mecanismos de Estructuración de Programas

Programación con Procedimientos

Basándose en los problemas de la programación sin modularización, surge la idea de separar las porciones de código que se necesitan reutilizar en procedimientos.

Básicamente, el programa principal se descompone en diferentes procedimientos. Éstos pueden ser invocados, lo cual provoca la ejecución del cuerpo del procedimiento, y el posterior retorno a la "posición siguiente" a la correspondiente a la invocación.

Gráficamente, la situación es la siguiente:



Mecanismos de Estructuración de Programas

Programación con Procedimientos

Mecanismos de Estructuración de Programas

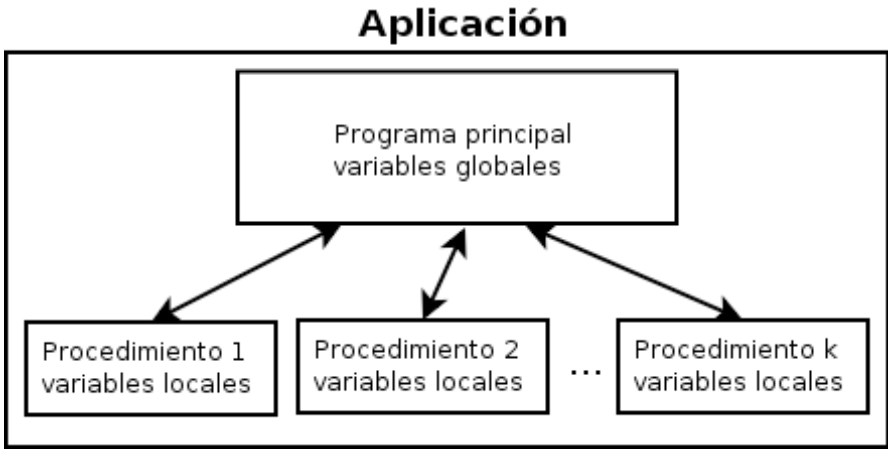
Programación con Procedimientos

Existen varios grados de aprovechamiento de la programación procedural (uso de parametrización o no, uso de variables locales o globales, etc). Gráficamente, un sistema que hace uso de este mecanismo tiene el siguiente esquema:

Mecanismos de Estructuración de Programas

Programación con Procedimientos

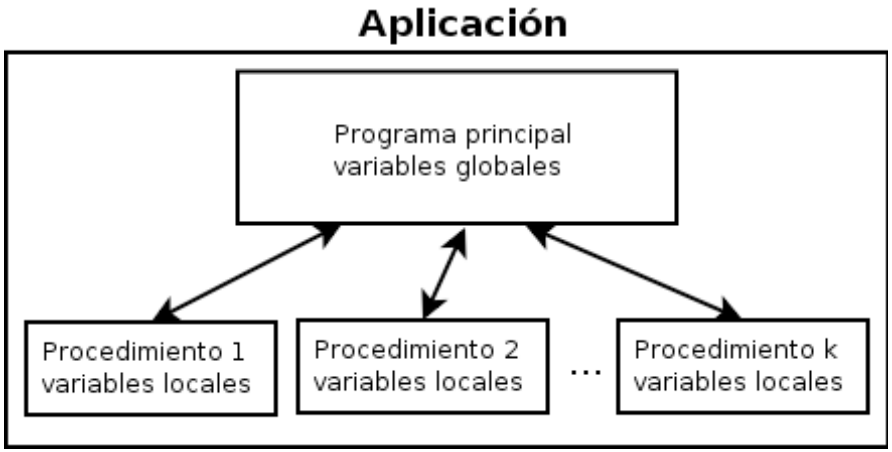
Existen varios grados de aprovechamiento de la programación procedural (uso de parametrización o no, uso de variables locales o globales, etc). Gráficamente, un sistema que hace uso de este mecanismo tiene el siguiente esquema:



Mecanismos de Estructuración de Programas

Programación con Procedimientos

Existen varios grados de aprovechamiento de la programación procedural (uso de parametrización o no, uso de variables locales o globales, etc). Gráficamente, un sistema que hace uso de este mecanismo tiene el siguiente esquema:



Asociados a este mecanismo, aparecen dos mecanismos de descomposición importantes, la abstracción por parametrización y la abstracción por especificación.

La programación con procedimientos da beneficios directos, ya que los errores son más fáciles de identificar y eliminar, el "espacio de nombres" es más manejable, etc.

Programación con Procedimientos

Abstracción por Especificación

Programación con Procedimientos

Abstracción por Especificación

La abstracción por especificación permite abstraernos de los detalles asociados a cómo el módulo (en este caso, procedimiento) resuelve un problema, y quedarnos con qué es lo que éste hace.

Programación con Procedimientos

Abstracción por Especificación

La abstracción por especificación permite abstraernos de los detalles asociados a cómo el módulo (en este caso, procedimiento) resuelve un problema, y quedarnos con qué es lo que éste hace.

Este tipo de abstracción nos permite separar la implementación de cada módulo/rutina.

Programación con Procedimientos

Abstracción por Especificación

La abstracción por especificación permite abstraernos de los detalles asociados a cómo el módulo (en este caso, procedimiento) resuelve un problema, y quedarnos con qué es lo que éste hace.

Este tipo de abstracción nos permite separar la implementación de cada módulo/rutina.

```
program Hello_World;

uses
    crt;

begin
    ClrScr;
    Write('Hello world');
    Readln;
end.
```

Programación con Procedimientos

Abstracción por Especificación

La abstracción por especificación permite abstraernos de los detalles asociados a cómo el módulo (en este caso, procedimiento) resuelve un problema, y quedarnos con qué es lo que éste hace.

Este tipo de abstracción nos permite separar la implementación de cada módulo/rutina.

```
program Hello_World;

uses
    crt;

begin
    ClrScr;
    Write('Hello world');
    Readln;
end.
```

Oculta los detalles de implementación de la rutina. Lo usamos pensando en su efecto, sin preocuparnos por su implementación

Programación con Procedimientos

Abstracción por Parametrización

Programación con Procedimientos

Abstracción por Parametrización

La abstracción por parametrización permite abstraernos de la identidad particular de algunos elementos usados en un programa, mediante la introducción de parámetros.

Programación con Procedimientos

Abstracción por Parametrización

La abstracción por parametrización permite abstraernos de la identidad particular de algunos elementos usados en un programa, mediante la introducción de parámetros.

Mediante la parametrización podemos, mediante un único programa, representar un conjunto potencialmente infinito de comportamientos (similares pero no exactamente iguales).

Programación con Procedimientos

Abstracción por Parametrización

La abstracción por parametrización permite abstraernos de la identidad particular de algunos elementos usados en un programa, mediante la introducción de parámetros.

Mediante la parametrización podemos, mediante un único programa, representar un conjunto potencialmente infinito de comportamientos (similares pero no exactamente iguales).

```
...
Writeln('****');
Writeln('Error');
Writeln('****');
Readln;
...
Writeln('****');
Writeln('Operacion exitosa');
Writeln('****');
Readln;
...
```

Programación con Procedimientos

Abstracción por Parametrización

La abstracción por parametrización permite abstraernos de la identidad particular de algunos elementos usados en un programa, mediante la introducción de parámetros.

Mediante la parametrización podemos, mediante un único programa, representar un conjunto potencialmente infinito de comportamientos (similares pero no exactamente iguales).

```
...
Writeln('****');
Writeln('Error');
Writeln('****');
Readln;
...
Writeln('****');
Writeln('Operacion exitosa');
Writeln('****');
Readln;
...
```

Programación con Procedimientos

Abstracción por Parametrización

La abstracción por parametrización permite abstraernos de la identidad particular de algunos elementos usados en un programa, mediante la introducción de parámetros.

Mediante la parametrización podemos, mediante un único programa, representar un conjunto potencialmente infinito de comportamientos (similares pero no exactamente iguales).

```
...
Writeln('****');
Writeln('Error');
Writeln('****');
Readln;
...
Writeln('****');
Writeln('Operacion exitosa');
Writeln('****');
Readln;
...
```

```
procedure imprimir_cartel(s: string);
begin
    Writeln('****');
    Writeln(s);
    Writeln('****');
    Readln;
end

...
imprimir_cartel('Error');
...
imprimir_cartel('Operacion exitosa');
...
```

Programación con Procedimientos

Abstracción por Parametrización

La abstracción por parametrización permite abstraernos de la identidad particular de algunos elementos usados en un programa, mediante la introducción de parámetros.

Mediante la parametrización podemos, mediante un único programa, representar un conjunto potencialmente infinito de comportamientos (similares pero no exactamente iguales).

```
...
Writeln('****');
Writeln('Error');
Writeln('****');
Readln;
...
Writeln('****');
Writeln('Operacion exitosa');
Writeln('****');
Readln;
...
```

```
procedure imprimir_cartel(s: string);
begin
    Writeln('****');
    Writeln(s);
    Writeln('****');
    Readln;
end

...
imprimir_cartel('Error');
...
imprimir_cartel('Operacion exitosa');
...
```

Mecanismos de Estructuración de Programas

Programación con Módulos

Mecanismos de Estructuración de Programas

Programación con Módulos

La programación con módulos agrega una nueva dimensión en la estructuración de programas, mediante el agrupamiento de procedimientos con funcionalidades comunes y los datos correspondientes en módulos (usualmente en archivos separados). Los programas consisten de varios módulos, con un módulo principal que dirige la interacción entre los módulos y el flujo de programa mediante llamadas a procedimientos.

Mecanismos de Estructuración de Programas

Programación con Módulos

La programación con módulos agrega una nueva dimensión en la estructuración de programas, mediante el agrupamiento de procedimientos con funcionalidades comunes y los datos correspondientes en módulos (usualmente en archivos separados). Los programas consisten de varios módulos, con un módulo principal que dirige la interacción entre los módulos y el flujo de programa mediante llamadas a procedimientos.

Cada módulo maneja su propios datos (variables locales al módulo). Cada módulo puede utilizarse a lo sumo una vez en el programa (es decir, no pueden tenerse más de una instancia de un módulo).

Mecanismos de Estructuración de Programas

Programación con Módulos

Mecanismos de Estructuración de Programas

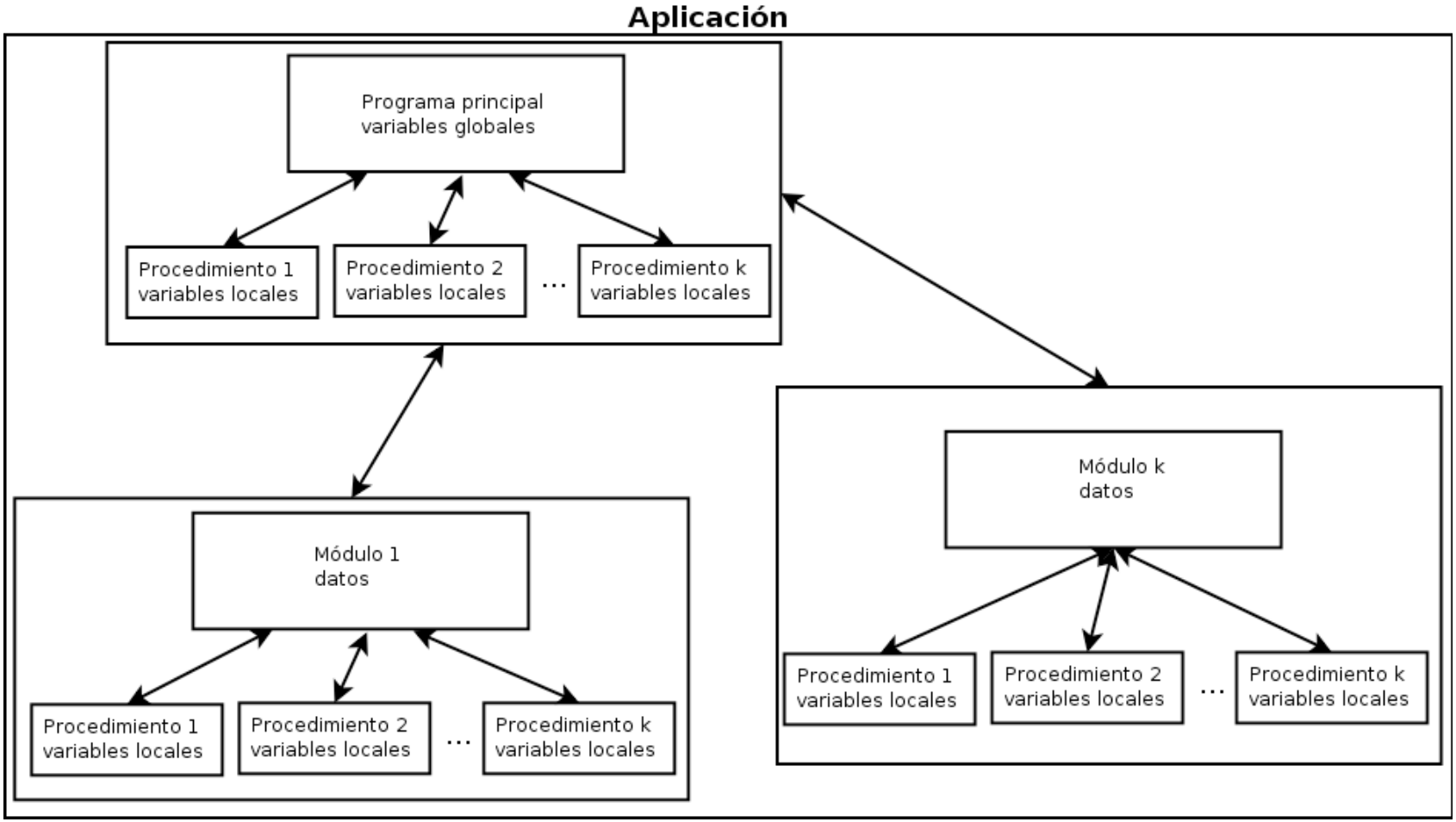
Programación con Módulos

La estructura de un programa organizado en términos de módulos es la siguiente:

Mecanismos de Estructuración de Programas

Programación con Módulos

La estructura de un programa organizado en términos de módulos es la siguiente:



Abstracción de Datos

Abstracción de Datos

Las abstracciones que vimos anteriormente corresponden a abstracciones procedimentales, es decir, descomposiciones basadas en la abstracción de procedimientos individuales. Este es un mecanismo de modularización muy poderoso, que nos permite extender la máquina virtual asociada al lenguaje de programación con nuevos procedimientos.

Abstracción de Datos

Las abstracciones que vimos anteriormente corresponden a abstracciones procedimentales, es decir, descomposiciones basadas en la abstracción de procedimientos individuales. Este es un mecanismo de modularización muy poderoso, que nos permite extender la máquina virtual asociada al lenguaje de programación con nuevos procedimientos.

Otro tipo de abstracción de gran utilidad es la abstracción de datos. La misma consiste en extender la máquina virtual asociada al lenguaje de programación con la definición de nuevos tipos de datos. La definición de tipos de datos en general requiere la definición de procedimientos para manejar estos tipos de datos.

Abstracción de Datos

Las abstracciones que vimos anteriormente corresponden a abstracciones procedimentales, es decir, descomposiciones basadas en la abstracción de procedimientos individuales. Este es un mecanismo de modularización muy poderoso, que nos permite extender la máquina virtual asociada al lenguaje de programación con nuevos procedimientos.

Otro tipo de abstracción de gran utilidad es la abstracción de datos. La misma consiste en extender la máquina virtual asociada al lenguaje de programación con la definición de nuevos tipos de datos. La definición de tipos de datos en general requiere la definición de procedimientos para manejar estos tipos de datos.

```
ptype
    pStack = ^tStack;
    tStack = record
        data: integer;
        prev: pStack;
    end;
```

Abstracción de Datos

Las abstracciones que vimos anteriormente corresponden a abstracciones procedimentales, es decir, descomposiciones basadas en la abstracción de procedimientos individuales. Este es un mecanismo de modularización muy poderoso, que nos permite extender la máquina virtual asociada al lenguaje de programación con nuevos procedimientos.

Otro tipo de abstracción de gran utilidad es la abstracción de datos. La misma consiste en extender la máquina virtual asociada al lenguaje de programación con la definición de nuevos tipos de datos. La definición de tipos de datos en general requiere la definición de procedimientos para manejar estos tipos de datos.

```
ptype
    pStack = ^tStack;
    tStack = record
        data: integer;
        prev: pStack;
    end;
```

```
procedure push(var s: pstack, i: integer);
cur: pStack;
begin
    new(cur);
    cur^.data := i;
    cur^.prev := s;
    s := cur;
end;
```

Abstracción de Datos en Lenguajes Imperativos (no OO)

Abstracción de Datos en Lenguajes Imperativos (no OO)

Los lenguajes de programación imperativa, no orientados a objetos, ofrecen una variedad de mecanismos para la definición de nuevos tipos de datos.

Abstracción de Datos en Lenguajes Imperativos (no OO)

Los lenguajes de programación imperativa, no orientados a objetos, ofrecen una variedad de mecanismos para la definición de nuevos tipos de datos.

Sin embargo, los mecanismos presentes en lenguajes imperativos sufren de varias limitaciones, tales como:

Abstracción de Datos en Lenguajes Imperativos (no OO)

Los lenguajes de programación imperativa, no orientados a objetos, ofrecen una variedad de mecanismos para la definición de nuevos tipos de datos.

Sin embargo, los mecanismos presentes en lenguajes imperativos sufren de varias limitaciones, tales como:

- creación y eliminación de instancias no uniforme

Abstracción de Datos en Lenguajes Imperativos (no OO)

Los lenguajes de programación imperativa, no orientados a objetos, ofrecen una variedad de mecanismos para la definición de nuevos tipos de datos.

Sin embargo, los mecanismos presentes en lenguajes imperativos sufren de varias limitaciones, tales como:

- creación y eliminación de instancias no uniforme
- dificultad en la definición de estructuras genéricas

Abstracción de Datos en Lenguajes Imperativos (no OO)

Los lenguajes de programación imperativa, no orientados a objetos, ofrecen una variedad de mecanismos para la definición de nuevos tipos de datos.

Sin embargo, los mecanismos presentes en lenguajes imperativos sufren de varias limitaciones, tales como:

- creación y eliminación de instancias no uniforme
- dificultad en la definición de estructuras genéricas
- datos separados de las operaciones en los mecanismos de definición de tipos

Características Fundamentales de la Programación “Procedimental”

Características Fundamentales de la Programación "Procedimental"

- El software se organiza alrededor de la noción de procedimiento

Características Fundamentales de la Programación "Procedimental"

- El software se organiza alrededor de la noción de procedimiento
- El mecanismo fundamental para lidiar con la complejidad es la abstracción procedimental

Características Fundamentales de la Programación "Procedimental"

- El software se organiza alrededor de la noción de procedimiento
- El mecanismo fundamental para lidiar con la complejidad es la abstracción procedimental
  - Funciona bien siempre y cuando los datos sean "simples", pues la modularidad viene sólo en la organización de procedimientos

Características Fundamentales de la Programación "Procedimental"

- El software se organiza alrededor de la noción de procedimiento
- El mecanismo fundamental para lidiar con la complejidad es la abstracción procedimental
  - Funciona bien siempre y cuando los datos sean "simples", pues la modularidad viene sólo en la organización de procedimientos
  - Los mecanismos de definición de abstracciones de datos son limitados (básicamente registros, tipos estructurados, etc.)

Características Fundamentales de la Programación Orientada a Objetos

Características Fundamentales de la Programación Orientada a Objetos

- Abstracción

Características Fundamentales de la Programación Orientada a Objetos

- Abstracción
  - provee mecanismos potentes para la definición de abstracciones de datos

Características Fundamentales de la Programación Orientada a Objetos

- Abstracción
  - provee mecanismos potentes para la definición de abstracciones de datos
- Encapsulamiento

Características Fundamentales de la Programación Orientada a Objetos

- Abstracción
  - provee mecanismos potentes para la definición de abstracciones de datos
- Encapsulamiento
  - una evolución de la metodología tradicional de separar las estructuras de datos de los algoritmos

Características Fundamentales de la Programación Orientada a Objetos

- Abstracción
  - provee mecanismos potentes para la definición de abstracciones de datos
- Encapsulamiento
  - una evolución de la metodología tradicional de separar las estructuras de datos de los algoritmos
  - Las operaciones (métodos) se asocian a las estructuras de datos en las cuales éstas operan

Características Fundamentales de la Programación Orientada a Objetos

- Abstracción
  - provee mecanismos potentes para la definición de abstracciones de datos
- Encapsulamiento
  - una evolución de la metodología tradicional de separar las estructuras de datos de los algoritmos
  - Las operaciones (métodos) se asocian a las estructuras de datos en las cuales éstas operan
  - si los módulos se definen apropiadamente, las estructuras de datos sólo pueden manipularse a través de las operaciones permitidas (ocultamiento)

Características Fundamentales de la Programación Orientada a Objetos

- Abstracción
  - provee mecanismos potentes para la definición de abstracciones de datos
- Encapsulamiento
  - una evolución de la metodología tradicional de separar las estructuras de datos de los algoritmos
  - Las operaciones (métodos) se asocian a las estructuras de datos en las cuales éstas operan
  - si los módulos se definen apropiadamente, las estructuras de datos sólo pueden manipularse a través de las operaciones permitidas (ocultamiento)

Si las abstracciones de datos son apropiadas, la modularidad y la reutilización pueden mejorarse considerablemente (respecto de la programación procedimental).

Clases y Procedimientos

Clases y Procedimientos

La Programación Orientada a Objetos está basada en la noción de objeto. En muchos lenguajes orientados a objetos, los objetos son instancias de clases.

Clases y Procedimientos

La Programación Orientada a Objetos está basada en la noción de objeto. En muchos lenguajes orientados a objetos, los objetos son instancias de clases.

Las clases son abstracciones de datos, que incluyen abstracciones procedimentales.

Clases y Procedimientos

La Programación Orientada a Objetos está basada en la noción de objeto. En muchos lenguajes orientados a objetos, los objetos son instancias de clases.

Las clases son abstracciones de datos, que incluyen abstracciones procedimentales.

Las clases comparten características con la noción de tipo abstracto de datos.

Clases y Procedimientos

La Programación Orientada a Objetos está basada en la noción de objeto. En muchos lenguajes orientados a objetos, los objetos son instancias de clases.

Las clases son abstracciones de datos, que incluyen abstracciones procedimentales.

Las clases comparten características con la noción de tipo abstracto de datos.

Si comparamos las organizaciones que la programación procedimental y POO favorecen, tendríamos por ejemplo lo siguiente:

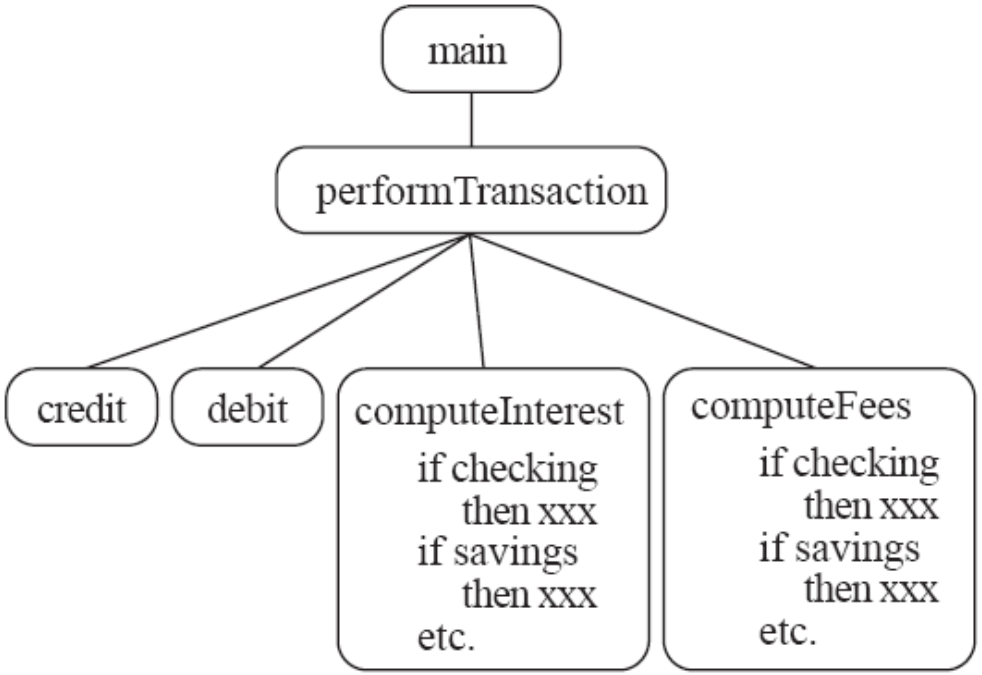
Clases y Procedimientos

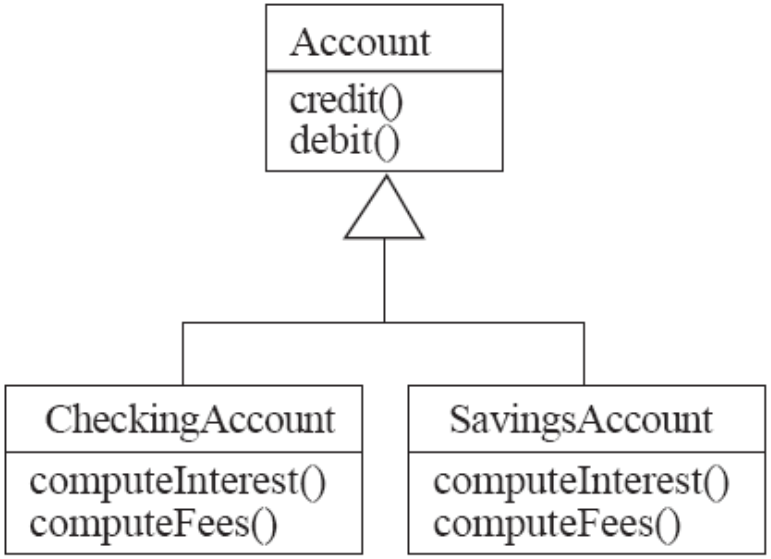
La Programación Orientada a Objetos está basada en la noción de objeto. En muchos lenguajes orientados a objetos, los objetos son instancias de clases.

Las clases son abstracciones de datos, que incluyen abstracciones procedimentales.

Las clases comparten características con la noción de tipo abstracto de datos.

Si comparamos las organizaciones que la programación procedimental y POO favorecen, tendríamos por ejemplo lo siguiente:





Conceptos Básicos de la Programación Orientada a Objetos

Conceptos Básicos de la Programación Orientada a Objetos

La mayoría de los lenguajes orientados modernos tienen:

Conceptos Básicos de la Programación Orientada a Objetos

La mayoría de los lenguajes orientados modernos tienen:

- tipos básicos (enteros, reales de punto flotante, caracteres, booleanos)

Conceptos Básicos de la Programación Orientada a Objetos

La mayoría de los lenguajes orientados modernos tienen:

- tipos básicos (enteros, reales de punto flotante, caracteres, booleanos)
- tipos estructurados (arreglos)

Conceptos Básicos de la Programación Orientada a Objetos

La mayoría de los lenguajes orientados modernos tienen:

- tipos básicos (enteros, reales de punto flotante, caracteres, booleanos)
- tipos estructurados (arreglos)
- punteros (o referencias)

Conceptos Básicos de la Programación Orientada a Objetos

La mayoría de los lenguajes orientados modernos tienen:

- tipos básicos (enteros, reales de punto flotante, caracteres, booleanos)
- tipos estructurados (arreglos)
- punteros (o referencias)
- sentencias procedimentales tradicionales, como asignación, iteración, composición condicional y composición secuencial

Conceptos Básicos de la Programación Orientada a Objetos

La mayoría de los lenguajes orientados modernos tienen:

- tipos básicos (enteros, reales de punto flotante, caracteres, booleanos)
- tipos estructurados (arreglos)
- punteros (o referencias)
- sentencias procedimentales tradicionales, como asignación, iteración, composición condicional y composición secuencial
- mecanismos avanzados de modularidad (clases)

Conceptos Básicos de la Programación Orientada a Objetos

La mayoría de los lenguajes orientados modernos tienen:

- tipos básicos (enteros, reales de punto flotante, caracteres, booleanos)
- tipos estructurados (arreglos)
- punteros (o referencias)
- sentencias procedimentales tradicionales, como asignación, iteración, composición condicional y composición secuencial
- mecanismos avanzados de modularidad (clases)
- mecanismos avanzados de reutilización (herencia, polimorfismo, genericidad)

La Noción de Clase

La Noción de Clase

Una clase es un mecanismo de modularización que encapsula:

La Noción de Clase

Una clase es un mecanismo de modularización que encapsula:

- la definición de estructuras de datos, mediante variables de otros tipos interrelacionadas (atributos o campos),

La Noción de Clase

Una clase es un mecanismo de modularización que encapsula:

- la definición de estructuras de datos, mediante variables de otros tipos interrelacionadas (atributos o campos),

- la definición de rutinas que operan sobre las estructuras de datos locales a la clase (métodos).

La Noción de Clase

Una clase es un mecanismo de modularización que encapsula:

- la definición de estructuras de datos, mediante variables de otros tipos interrelacionadas (atributos o campos),

- la definición de rutinas que operan sobre las estructuras de datos locales a la clase (métodos).

Los lenguajes de programación orientados a objetos poseen en general mecanismos para definir interfaces, es decir, de indicar cuáles de los elementos que constituyen una clase pueden utilizarse "desde afuera".

Un Ejemplo de Clases

Un Ejemplo de Clases

Consideremos el siguiente ejemplo simple de una clase (en pseudo-código)

Un Ejemplo de Clases

Consideremos el siguiente ejemplo simple de una clase (en pseudo-código)

```
clase Circulo
  atributos
    centro_x: real [privado]
    centro_y: real [privado]
    radio: real [privado]
```

Un Ejemplo de Clases

Consideremos el siguiente ejemplo simple de una clase (en pseudo-código)

```
clase Circulo
  atributos
    centro_x: real [privado]
    centro_y: real [privado]
    radio: real [privado]
  métodos
    centro() : (real,real) [público]
        retornar (centro_x, centro_y)
    fin
    radio() : real [público]
        retornar (radio)
    fin
```

Un Ejemplo de Clases

Consideremos el siguiente ejemplo simple de una clase (en pseudo-código)

```
clase Circulo
  atributos
    centro_x: real [privado]
    centro_y: real [privado]
    radio: real [privado]
  métodos
    centro() : (real,real) [público]
        retornar (centro_x, centro_y)
    fin
    radio() : real [público]
        retornar (radio)
    fin
    circunferencia() : real [público]
        retornar (radio*pi)
    fin
    nuevo_centro(x,y: real) [público]
        centro_x:=x; centro_y:=y
    fin
    nuevo_radio(r: real) [público]
        radio:=r
    fin
fin_clase
```

Las Clases Definen Tipos

Las Clases Definen Tipos

La definición de una clase es la definición de un nuevo tipo (además de la definición de un módulo).

Las Clases Definen Tipos

La definición de una clase es la definición de un nuevo tipo (además de la definición de un módulo).

Los atributos y métodos públicos definen la interfaz del tipo.

Las Clases Definen Tipos

La definición de una clase es la definición de un nuevo tipo (además de la definición de un módulo).

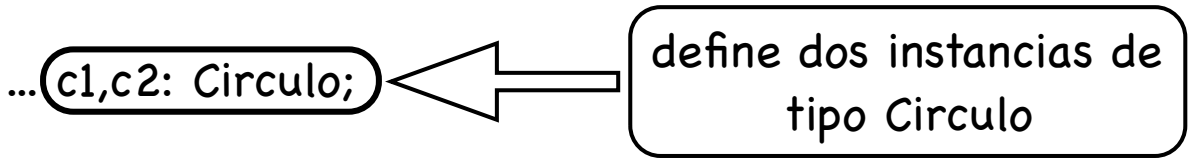
Los atributos y métodos públicos definen la interfaz del tipo.

```
... c1,c2: Circulo;

...

c1.nuevo_centro(1,5);

c1.nuevo_radio(10);

...

Si c2.radio() = 0 entonces c2.nuevo_radio(c1.radio());

...

c2.radio:=0;

c2.nuevo_radio(c1.radio);

...
```

Las Clases Definen Tipos

La definición de una clase es la definición de un nuevo tipo (además de la definición de un módulo).

Los atributos y métodos públicos definen la interfaz del tipo.

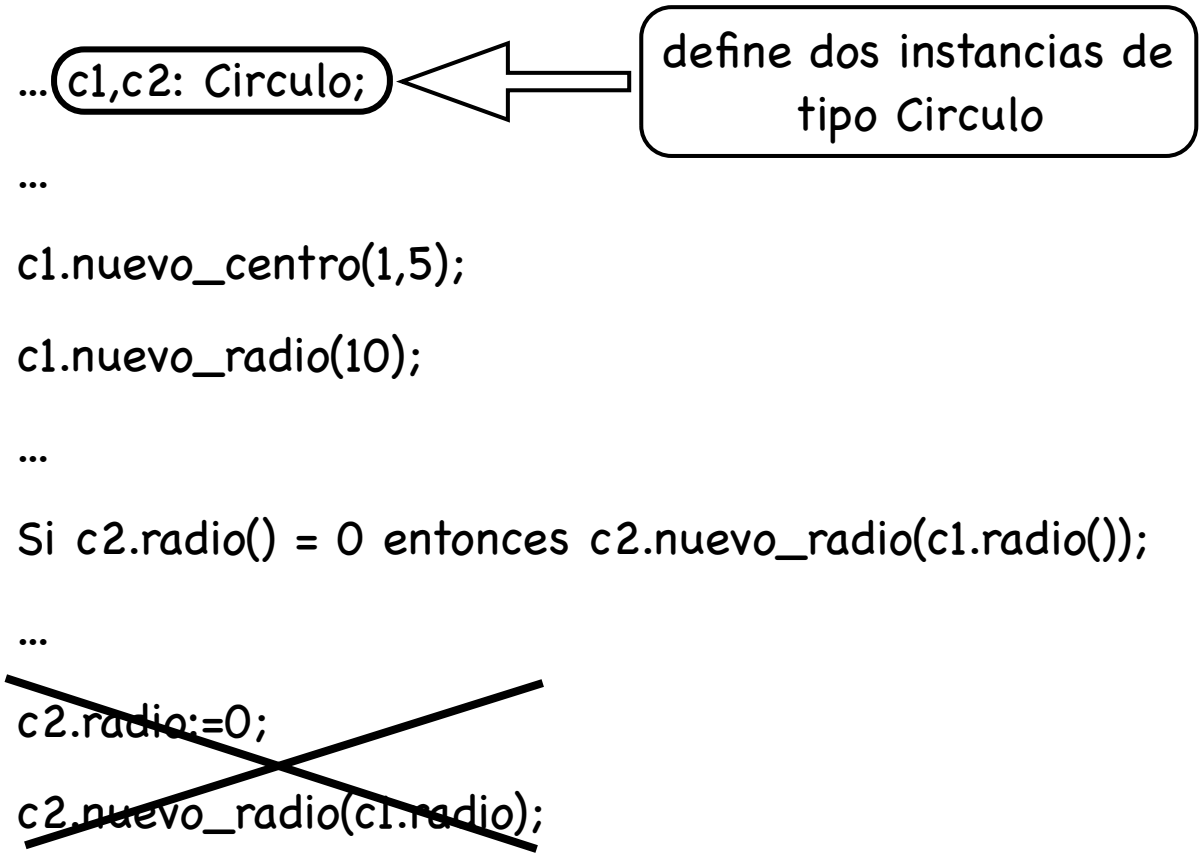



```
...
c1.nuevo_centro(1,5);
c1.nuevo_radio(10);
...
Si c2.radio() = 0 entonces c2.nuevo_radio(c1.radio());
...
c2.radio:=0;
c2.nuevo_radio(c1.radio);
...
```

Las Clases Definen Tipos

La definición de una clase es la definición de un nuevo tipo (además de la definición de un módulo).

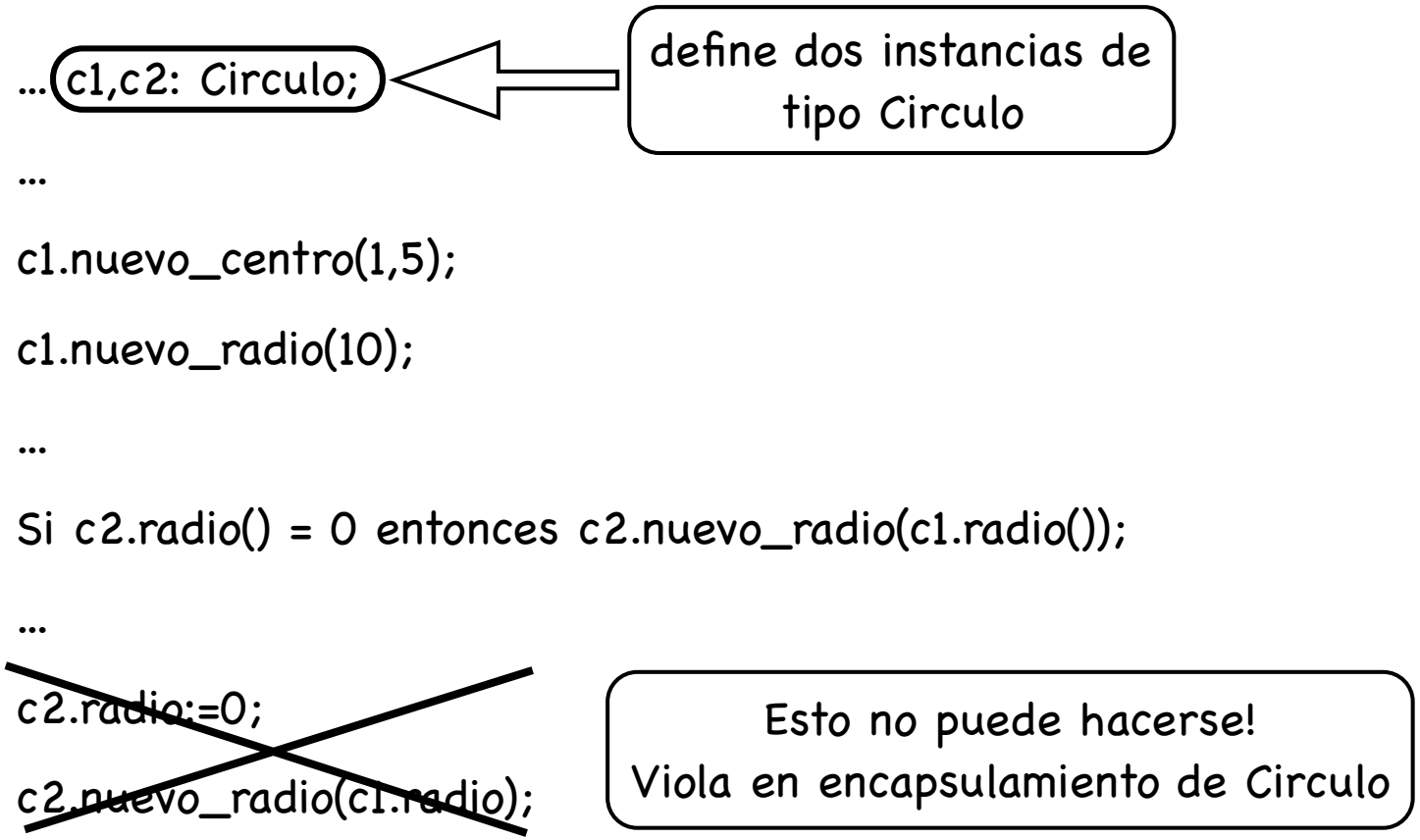
Los atributos y métodos públicos definen la interfaz del tipo.



Las Clases Definen Tipos

La definición de una clase es la definición de un nuevo tipo (además de la definición de un módulo).

Los atributos y métodos públicos definen la interfaz del tipo.



Múltiples Instancias

Múltiples Instancias

Como vimos en el ejemplo anterior, uno puede utilizar varias instancias de la misma clase. Las instancias se denominan objetos. Cada objeto tiene una identidad propia, más allá de los valores de las estructuras internas.

Múltiples Instancias

Como vimos en el ejemplo anterior, uno puede utilizar varias instancias de la misma clase. Las instancias se denominan objetos. Cada objeto tiene una identidad propia, más allá de los valores de las estructuras internas.

Por ejemplo, si los círculos c1 y c2 tienen el mismo radio y el mismo centro, esto no los hace necesariamente el mismo objeto! (de la misma forma que, si dos variables enteras diferentes tienen el mismo valor almacenado, las variables siguen siendo distintas).

Múltiples Instancias

Como vimos en el ejemplo anterior, uno puede utilizar varias instancias de la misma clase. Las instancias se denominan objetos. Cada objeto tiene una identidad propia, más allá de los valores de las estructuras internas.

Por ejemplo, si los círculos c1 y c2 tienen el mismo radio y el mismo centro, esto no los hace necesariamente el mismo objeto! (de la misma forma que, si dos variables enteras diferentes tienen el mismo valor almacenado, las variables siguen siendo distintas).

```
...
c1,c2: Circulo;
...
c1.nuevo_centro(1,5);
c1.nuevo_radio(10);
c2.nuevo_centro(1,5);
c2.nuevo_radio(10);
```

Múltiples Instancias

Como vimos en el ejemplo anterior, uno puede utilizar varias instancias de la misma clase. Las instancias se denominan objetos. Cada objeto tiene una identidad propia, más allá de los valores de las estructuras internas.

Por ejemplo, si los círculos c1 y c2 tienen el mismo radio y el mismo centro, esto no los hace necesariamente el mismo objeto! (de la misma forma que, si dos variables enteras diferentes tienen el mismo valor almacenado, las variables siguen siendo distintas).

```
...
c1,c2: Circulo;
...
c1.nuevo_centro(1,5);
c1.nuevo_radio(10);
c2.nuevo_centro(1,5);
c2.nuevo_radio(10);
```

c1 y c2 pueden ser dos objetos distintos!

Constructores y Destructores

Constructores y Destructores

La inicialización y destrucción de objetos se realiza de forma homogénea, a través de métodos especiales de la clase correspondiente llamados constructores y destructores:

Constructores y Destructores

La inicialización y destrucción de objetos se realiza de forma homogénea, a través de métodos especiales de la clase correspondiente llamados constructores y destructores:

```
clase Circulo
  Circulo() [público]
      centro_x:= 0;
      centro_y:= 0;
      radio:= 0
  fin
  ~Circulo() [publico]
        skip
    fin
```

Constructores y Destructores

La inicialización y destrucción de objetos se realiza de forma homogénea, a través de métodos especiales de la clase correspondiente llamados constructores y destructores:

```
clase Circulo
  Circulo() [público]
      centro_x:= 0;
      centro_y:= 0;
      radio:= 0
  fin
  ~Circulo() [publico]
      skip
  fin
```

cuando se usan estructuras dinámicas, debemos "destruir" o liberar las mismas aquí

Constructores y Destructores

Invocación explícita del constructor

Constructores y Destructores

Invocación explícita del constructor

Los clientes invocan al constructor explícitamente:

Constructores y Destructores

Invocación explícita del constructor

Los clientes invocan al constructor explícitamente:

```
c1: Circulo

...

c1:= nuevo Circulo();

...
```

Constructores y Destructores

Invocación explícita del constructor

Los clientes invocan al constructor explícitamente:

```
c1: Circulo

...

c1:= nuevo Circulo();

...
```

La invocación al constructor tiene por efecto:

Constructores y Destructores

Invocación explícita del constructor

Los clientes invocan al constructor explícitamente:

```
c1: Circulo

...

c1:= nuevo Circulo();

...
```

La invocación al constructor tiene por efecto:

- alojar espacio de almacenamiento para los atributos de la clase

Constructores y Destructores

Invocación explícita del constructor

Los clientes invocan al constructor explícitamente:

```
c1: Circulo

...

c1:= nuevo Circulo();

...
```

La invocación al constructor tiene por efecto:

- alojar espacio de almacenamiento para los atributos de la clase
- ejecutar el código del cuerpo del constructor (e.g., inicialización de atributos)

Aplicaciones en el Paradigma Orientado a Objetos

Aplicaciones en el Paradigma Orientado a Objetos

En la programación orientada a objetos se programa modularmente usando clases. Una aplicación es una colección de clases, en la cual una clase es identificada como la principal. Esta clase contiene un método principal, que es el que se ejecuta al iniciar la aplicación.

Aplicaciones en el Paradigma Orientado a Objetos

En la programación orientada a objetos se programa modularmente usando clases. Una aplicación es una colección de clases, en la cual una clase es identificada como la principal. Esta clase contiene un método principal, que es el que se ejecuta al iniciar la aplicación.

```
clase Principal
  métodos
      principal [público]
          c1,c2: Circulo;
          ...
          c1:= nuevo Circulo();
          c1.nuevo_radio(10);
          ...
      fin
```

Aplicaciones en el Paradigma Orientado a Objetos

En la programación orientada a objetos se programa modularmente usando clases. Una aplicación es una colección de clases, en la cual una clase es identificada como la principal. Esta clase contiene un método principal, que es el que se ejecuta al iniciar la aplicación.

```
clase Principal
  métodos
      principal [público]
          c1,c2: Circulo;
          ...
          c1:= nuevo Circulo();
          c1.nuevo_radio(10);
          ...
      fin
```

Sólo puede existir una instancia de la clase principal!

Programación Orientada a Objetos y Definición de TADs

Programación Orientada a Objetos y Definición de TADs

La tarea de programar en un lenguaje OO consiste en identificar y definir abstracciones de datos, que resulten útiles para resolver el problema particular de programación que se tenga.

Programación Orientada a Objetos y Definición de TADs

La tarea de programar en un lenguaje OO consiste en identificar y definir abstracciones de datos, que resulten útiles para resolver el problema particular de programación que se tenga.

Los lenguajes OO ofrecen una forma natural de definir TADs y sus implementaciones: las clases. Es importante entonces tener en cuenta que:

Programación Orientada a Objetos y Definición de TADs

La tarea de programar en un lenguaje OO consiste en identificar y definir abstracciones de datos, que resulten útiles para resolver el problema particular de programación que se tenga.

Los lenguajes OO ofrecen una forma natural de definir TADs y sus implementaciones: las clases. Es importante entonces tener en cuenta que:

- Los TADs agrupan los datos con las operaciones permitidas sobre éstos.

Programación Orientada a Objetos y Definición de TADs

La tarea de programar en un lenguaje OO consiste en identificar y definir abstracciones de datos, que resulten útiles para resolver el problema particular de programación que se tenga.

Los lenguajes OO ofrecen una forma natural de definir TADs y sus implementaciones: las clases. Es importante entonces tener en cuenta que:

- Los TADs agrupan los datos con las operaciones permitidas sobre éstos.

- Debe separarse el propósito de los TADs de sus implementaciones (abstracción).

Programación Orientada a Objetos y Definición de TADs

La tarea de programar en un lenguaje OO consiste en identificar y definir abstracciones de datos, que resulten útiles para resolver el problema particular de programación que se tenga.

Los lenguajes OO ofrecen una forma natural de definir TADs y sus implementaciones: las clases. Es importante entonces tener en cuenta que:

- Los TADs agrupan los datos con las operaciones permitidas sobre éstos.

- Debe separarse el propósito de los TADs de sus implementaciones (abstracción).

- Deben ocultarse los detalles sobre la implementación de TADs que los "clientes" no necesitan (o no deben) conocer (encapsulamiento).

Programación Orientada a Objetos y Definición de TADs

La tarea de programar en un lenguaje OO consiste en identificar y definir abstracciones de datos, que resulten útiles para resolver el problema particular de programación que se tenga.

Los lenguajes OO ofrecen una forma natural de definir TADs y sus implementaciones: las clases. Es importante entonces tener en cuenta que:

- Los TADs agrupan los datos con las operaciones permitidas sobre éstos.

- Debe separarse el propósito de los TADs de sus implementaciones (abstracción).

- Deben ocultarse los detalles sobre la implementación de TADs que los "clientes" no necesitan (o no deben) conocer (encapsulamiento).

- A través de mecanismos simples, debe permitirse cambiar la representación de datos (por ejemplo, con implementaciones más eficientes) asociada a un TAD.

Ejemplo: El Lenguaje Java

Ejemplo: El Lenguaje Java

Java es un lenguaje de programación orientado a objetos, con las siguientes características básicas:

Ejemplo: El Lenguaje Java

Java es un lenguaje de programación orientado a objetos, con las siguientes características básicas:

- tipos básicos: int, float, double, boolean, char, ...

Ejemplo: El Lenguaje Java

Java es un lenguaje de programación orientado a objetos, con las siguientes características básicas:

- tipos básicos: int, float, double, boolean, char, ...
- clases, referencias, herencia (simple), polimorfismo, genericidad

Ejemplo: El Lenguaje Java

Java es un lenguaje de programación orientado a objetos, con las siguientes características básicas:

- tipos básicos: int, float, double, boolean, char, ...
- clases, referencias, herencia (simple), polimorfismo, genericidad
- gran variedad de clases predefinidas

Ejemplo: El Lenguaje Java

Java es un lenguaje de programación orientado a objetos, con las siguientes características básicas:

- tipos básicos: int, float, double, boolean, char, ...
- clases, referencias, herencia (simple), polimorfismo, genericidad
- gran variedad de clases predefinidas
- garbage collection

Ejemplo: El Lenguaje Java

Java es un lenguaje de programación orientado a objetos, con las siguientes características básicas:

- tipos básicos: int, float, double, boolean, char, ...
- clases, referencias, herencia (simple), polimorfismo, genericidad
- gran variedad de clases predefinidas
- garbage collection
- portabilidad a nivel de código objeto

Ejemplo (Procedimental) en Java

Ejemplo (Procedimental) en Java

```
public class cuadrado {

    public static void main(String[] args) {

        if (args.length != 1) {
            System.out.println("*** Uso: java Cuadrado <numero>");
        }
        else {
            int input = Integer.parseInt(args[0]);
            int res = input*input;
            System.out.println("El cuadrado de " + input + " es: " + res);
        }
    } // fin de main

} // fin de clase cuadrado
```

Ejemplo de Abstracciones de Datos en Java

Ejemplo de Abstracciones de Datos en Java

```
public class IntPair {
    private int x;
    private int y;

    public IntPair() {
      x = 0;
      y = 0;
    }

    public IntPair(int v1, int v2) {
      x = v1;
      y = v2;
    }
```

Ejemplo de Abstracciones de Datos en Java

```
public class IntPair {
    private int x;
    private int y;

    public IntPair() {
      x = 0;
      y = 0;
    }

    public IntPair(int v1, int v2) {
      x = v1;
      y = v2;
    }
```

```
public void changeFst(int v1) {
      x = v1;
    }

    public void changeSnd(int v2) {
      y = v2;
    }

    public int fst() {
      return x;
    }

    public int snd() {
      return y;
    }

}
```

Uso de Abstracciones de Datos

Uso de Abstracciones de Datos

```
public static IntPair minMaxArray(int[] A, int i, int j) {

  if (i>j) {
      IntPair res = new IntPair(0,0);
      return res; // special case: we return (0,0) if the array is empty
  }
  else {
      IntPair res_sub = new IntPair(A[i],A[i]);
      int k = i;
      while (k<=j) {
          if (A[k]<res_sub.fst()) {
               res_sub.changeFst(A[k]);
           }
          if (A[k]>res_sub.snd()) {
               res_sub.changeSnd(A[k]);
           }
           k++;
      }
      return res_sub;
  }
} // end of minMaxArray
```

Uso de Abstracciones de Datos (cont.)

Uso de Abstracciones de Datos (cont.)

```
public static IntPair rMinMaxArray(int[] A, int i, int j) {
 if (i>j) {
     IntPair res = new IntPair(0,0);
     return res; // special case: we return (0,0) if the array is empty
 }
 else {
     if (i==j) {
         IntPair res = new IntPair(A[i],A[i]);
         return res; // base case: the array under consideration has size 1
     }
     else {
         IntPair res_sub = new IntPair();
         IntPair res = new IntPair();
         res_sub = rMinMaxArray(A, i+1, j); // we calculate the min & max of subarray i+1 j
         if (res_sub.fst()>A[i]) {
              res.changeFst(A[i]);
         }
         else {
             res.changeFst(res_sub.fst());
         }
         if (res_sub.snd()<A[i]) {
             res.changeSnd(A[i]);
         }
         else {
             res.changeSnd(res_sub.snd());
         }
         return res;
     }
 }
} // end of rMinMaxArray
```

Desarrollo Orientado a Objetos

Desarrollo Orientado a Objetos

La orientación a objetos es una metodología de diseño de software basada fuertemente en la noción de tipo abstractos de datos.

Desarrollo Orientado a Objetos

La orientación a objetos es una metodología de diseño de software basada fuertemente en la noción de tipo abstractos de datos.

La orientación a objetos puede verse de alguna manera como una evolución de técnicas de programación previas, en la cual:

Desarrollo Orientado a Objetos

La orientación a objetos es una metodología de diseño de software basada fuertemente en la noción de tipo abstractos de datos.

La orientación a objetos puede verse de alguna manera como una evolución de técnicas de programación previas, en la cual:

- se utilizan módulos para encapsular datos y comportamiento,

Desarrollo Orientado a Objetos

La orientación a objetos es una metodología de diseño de software basada fuertemente en la noción de tipo abstractos de datos.

La orientación a objetos puede verse de alguna manera como una evolución de técnicas de programación previas, en la cual:

- se utilizan módulos para encapsular datos y comportamiento,

- se combinan la noción de definición de módulos con la noción de definición de tipos,

Desarrollo Orientado a Objetos

La orientación a objetos es una metodología de diseño de software basada fuertemente en la noción de tipo abstractos de datos.

La orientación a objetos puede verse de alguna manera como una evolución de técnicas de programación previas, en la cual:

- se utilizan módulos para encapsular datos y comportamiento,
- se combinan la noción de definición de módulos con la noción de definición de tipos,
- se combinan la categorización de entidades (tipado) con la extensión de módulos.

Desarrollo Orientado a Objetos

La orientación a objetos es una metodología de diseño de software basada fuertemente en la noción de tipo abstractos de datos.

La orientación a objetos puede verse de alguna manera como una evolución de técnicas de programación previas, en la cual:

- se utilizan módulos para encapsular datos y comportamiento,
- se combinan la noción de definición de módulos con la noción de definición de tipos,
- se combinan la categorización de entidades (tipado) con la extensión de módulos.

Si se utiliza correctamente, la orientación a objetos puede contribuir a la reutilización, la facilidad de modificación y de depuración de software.

Relaciones entre Clases

Relaciones entre Clases

En Programación orientada a objetos, existen esencialmente dos relaciones posibles entre clases:

Relaciones entre Clases

En Programación orientada a objetos, existen esencialmente dos relaciones posibles entre clases:

- una clase puede ser cliente de otra (relación de uso),

Relaciones entre Clases

En Programación orientada a objetos, existen esencialmente dos relaciones posibles entre clases:

- una clase puede ser cliente de otra (relación de uso),
- una clase puede extender o especializar otra clase.

Relaciones entre Clases

En Programación orientada a objetos, existen esencialmente dos relaciones posibles entre clases:

- una clase puede ser cliente de otra (relación de uso),
- una clase puede extender o especializar otra clase.

Este segundo tipo de relación es la correspondiente a herencia. Si las clases se ven como módulos, entonces herencia corresponde a extensión (el módulo que extiende provee nuevas funcionalidades). Si las clases se ven como definiciones de tipo, entonces la clase que extiende define un tipo más especial.

Herencia de Clases en Java

Herencia de Clases en Java

Podemos hacer que una clase herede la estructura de otra mediante la palabra reservada "extends".

Herencia de Clases en Java

Podemos hacer que una clase herede la estructura de otra mediante la palabra reservada "extends".

Ejemplo: Supongamos que queremos extender la clase IntPair con un método que permita intercambiar los valores de x e y. Podemos hacerlo definiendo una nueva clase, de la siguiente manera:

Herencia de Clases en Java

Podemos hacer que una clase herede la estructura de otra mediante la palabra reservada "extends".

Ejemplo: Supongamos que queremos extender la clase IntPair con un método que permita intercambiar los valores de x e y. Podemos hacerlo definiendo una nueva clase, de la siguiente manera:

```
public class SwappableIntPair extends IntPair {

    public void swap() {
        int aux = fst();
        changeFst(snd());
        changeSnd(aux);
    }

} // fin de clase SwappableIntPair
```

Herencia de Clases en Java

Podemos hacer que una clase herede la estructura de otra mediante la palabra reservada "extends".

Ejemplo: Supongamos que queremos extender la clase IntPair con un método que permita intercambiar los valores de x e y. Podemos hacerlo definiendo una nueva clase, de la siguiente manera:

```
public class SwappableIntPair extends IntPair {

    public void swap() {
        int aux = fst();
        changeFst(snd());
        changeSnd(aux);
    }

} // fin de clase SwappableIntPair
```

Hereda todas las definiciones (atributos, métodos y su visibilidad) de IntPair

Herencia como Extensión de Módulos

Herencia como Extensión de Módulos

Al definir una clase Sub como subclase de una clase Super, indicamos que Sub extiende, como módulo, a Super., es decir, heredamos su estructura (métodos y atributos).

Herencia como Extensión de Módulos

Al definir una clase Sub como subclase de una clase Super, indicamos que Sub extiende, como módulo, a Super., es decir, heredamos su estructura (métodos y atributos).

Consideremos el siguiente ejemplo de uso de SwappableIntPair:

Herencia como Extensión de Módulos

Al definir una clase Sub como subclase de una clase Super, indicamos que Sub extiende, como módulo, a Super., es decir, heredamos su estructura (métodos y atributos).

Consideremos el siguiente ejemplo de uso de SwappableIntPair:

```
public class fibonacci {

    public static void main(String[] args) {

        SwappableIntPair p = new SwappableIntPair(1,1);
        int n = Integer.parseInt(args[0]);
        for (int i=1;i<=n;i++) {
            p = siguienteFib(p);
        }
        System.out.println(p.snd());

   } // end of main

   public static SwappableIntPair siguienteFib(SwappableIntPair p) {

        int siguiente = p.fst() + p.snd();
        p.swap();
        p.changeSnd(siguiente);
        return p;
   }

} // end of fibonacci
```

Herencia como Extensión de Módulos

Al definir una clase Sub como subclase de una clase Super, indicamos que Sub extiende, como módulo, a Super., es decir, heredamos su estructura (métodos y atributos).

Consideremos el siguiente ejemplo de uso de SwappableIntPair:

```
public class fibonacci {

    public static void main(String[] args) {

        SwappableIntPair p = new SwappableIntPair(1,1);
        int n = Integer.parseInt(args[0]);
        for (int i=1;i<=n;i++) {
            p = siguienteFib(p);
        }
        System.out.println(p.snd());

   } // end of main

   public static SwappableIntPair siguienteFib(SwappableIntPair p) {

        int siguiente = p.fst() + p.snd();
        p.swap();
        p.changeSnd(siguiente);
        return p;
   }

} // end of fibonacci
```

Herencia como Extensión de Tipos

Herencia como Extensión de Tipos

Al definir una clase Sub como subclase de una clase Super, además de indicar que Sub extiende, como módulo, a Super, indicamos que Sub es un subtipo de Super. De esta manera, cualquier instancia de Sub es también una instancia de Super.

Herencia como Extensión de Tipos

Al definir una clase Sub como subclase de una clase Super, además de indicar que Sub extiende, como módulo, a Super, indicamos que Sub es un subtipo de Super. De esta manera, cualquier instancia de Sub es también una instancia de Super.

```
public class IntPair {
    private int x;
    private int y;

    ...

  public boolean equals(IntPair other) {
       return (x==other.fst() && y==other.snd());
  }

}
```

Herencia como Extensión de Tipos

Al definir una clase Sub como subclase de una clase Super, además de indicar que Sub extiende, como módulo, a Super, indicamos que Sub es un subtipo de Super. De esta manera, cualquier instancia de Sub es también una instancia de Super.

```
public class IntPair {
    private int x;
    private int y;

    ...

  public boolean equals(IntPair other) {
       return (x==other.fst() && y==other.snd());
  }

}
```

```
public class paresIguales {

    public static void main(String[] args) {

        SwappableIntPair p1 = new SwappableIntPair(1,1);
        IntPair p2 = new IntPair(1,1);
        System.out.println(p2.equals(p1));
    }

} // end of paresIguales
```

Herencia como Extensión de Tipos

Al definir una clase Sub como subclase de una clase Super, además de indicar que Sub extiende, como módulo, a Super, indicamos que Sub es un subtipo de Super. De esta manera, cualquier instancia de Sub es también una instancia de Super.

```
public class IntPair {
    private int x;
    private int y;

    ...

  public boolean equals(IntPair other) {
        return (x==other.fst() && y==other.snd());
  }

}
```

```
public class paresIguales {

    public static void main(String[] args) {

        SwappableIntPair p1 = new SwappableIntPair(1,1);
        IntPair p2 = new IntPair(1,1);
        System.out.println(p2.equals(p1));
    }

} // end of paresIguales
```

Herencia como Extensión de Tipos

Al definir una clase Sub como subclase de una clase Super, además de indicar que Sub extiende, como módulo, a Super, indicamos que Sub es un subtipo de Super. De esta manera, cualquier instancia de Sub es también una instancia de Super.

```
public class IntPair {
    private int x;
    private int y;

    ...

  public boolean equals(IntPair other) {
        return (x==other.fst() && y==other.snd());
  }

}
```

```
public class paresIguales {

    public static void main(String[] args) {

        SwappableIntPair p1 = new SwappableIntPair(1,1);
        IntPair p2 = new IntPair(1,1);
        System.out.println(p2.equals(p1));
    }

} // end of paresIguales
```

equals espera un IntPair, y le pasamos un SwappableIntPair!

Herencia y Polimorfismo

Herencia y Polimorfismo

La herencia en lenguajes orientados a objetos está fuertemente relacionada a la noción de polimorfismo. Un método que toma como parámetro un objeto de tipo X es polimórfico, en el sentido de que puede aplicarse a cualquier objeto de clase Y, con Y subclase de X.

Herencia y Polimorfismo

La herencia en lenguajes orientados a objetos está fuertemente relacionada a la noción de polimorfismo. Un método que toma como parámetro un objeto de tipo X es polimórfico, en el sentido de que puede aplicarse a cualquier objeto de clase Y, con Y subclase de X.

Ejemplo: La función equals puede aplicarse a instancias de IntPair o instancias de SwappableIntPair. Más aún, si definimos subclases de SwappableIntPair, la función equals podrá aplicarse a todas ellas.

Redefinición de Métodos

Redefinición de Métodos

Al utilizar herencia para extender módulos, uno puede necesitar (y esto sucede con frecuencia) redefinir algunos de los métodos definidos en la super clase. Esto produce que un mismo método posea diferentes definiciones asociadas al mismo (una para la raíz de la jerarquía, y una por cada subclase que redefina el método).

Redefinición de Métodos

Al utilizar herencia para extender módulos, uno puede necesitar (y esto sucede con frecuencia) redefinir algunos de los métodos definidos en la super clase. Esto produce que un mismo método posea diferentes definiciones asociadas al mismo (una para la raíz de la jerarquía, y una por cada subclase que redefina el método).

```
public class MemoryIntPair extends IntPair {

    private int previousX;
    private int previousY;

    public MemoryIntPair() {
       super();
    }

    public MemoryIntPair(int v1, int v2) {
        super(v1,v2);
    }

    public void changeFst(int v1) {
       previousX = fst();
        super.changeFst(v1);
   }

   public void changeSnd(int v2) {
       previousY = snd();
        super.changeSnd(v2);
   }

   public void recover() {
       changeFst(previousX);
        changeSnd(previousY);
   }

}
```

Redefinición de Métodos

Al utilizar herencia para extender módulos, uno puede necesitar (y esto sucede con frecuencia) redefinir algunos de los métodos definidos en la super clase. Esto produce que un mismo método posea diferentes definiciones asociadas al mismo (una para la raíz de la jerarquía, y una por cada subclase que redefina el método).

```
public class MemoryIntPair extends IntPair {

    private int previousX;
    private int previousY;

    public MemoryIntPair() {
       super();
    }

    public MemoryIntPair(int v1, int v2) {
        super(v1,v2);
    }

    public void changeFst(int v1) {
       previousX = fst();
        super.changeFst(v1);
   }

   public void changeSnd(int v2) {
       previousY = snd();
        super.changeSnd(v2);
   }

   public void recover() {
       changeFst(previousX);
        changeSnd(previousY);
   }

}
```

Redefinición de Métodos

Al utilizar herencia para extender módulos, uno puede necesitar (y esto sucede con frecuencia) redefinir algunos de los métodos definidos en la super clase. Esto produce que un mismo método posea diferentes definiciones asociadas al mismo (una para la raíz de la jerarquía, y una por cada subclase que redefina el método).

```
public class MemoryIntPair extends IntPair {

    private int previousX;
    private int previousY;

    public MemoryIntPair() {
       super();
    }

    public MemoryIntPair(int v1, int v2) {
        super(v1,v2);
    }

    public void changeFst(int v1) {
       previousX = fst();
        super.changeFst(v1);
    }

    public void changeSnd(int v2) {
       previousY = snd();
        super.changeSnd(v2);
    }

    public void recover() {
       changeFst(previousX);
        changeSnd(previousY);
    }

}
```

Redefinición de Métodos

Redefinición de Métodos

Los lenguajes orientados a objetos poseen un mecanismo para "invocar" el método correcto en tiempo de ejecución, denominado dynamic dispatch.

Redefinición de Métodos

Los lenguajes orientados a objetos poseen un mecanismo para "invocar" el método correcto en tiempo de ejecución, denominado dynamic dispatch.

```
public class paresIgualesMem {

    public static void main(String[] args) {

       SwappableIntPair p1 = new SwappableIntPair();
       MemoryIntPair p2 = new MemoryIntPair();
       p1.changeFst(1);
       p2.changeFst(2);
       System.out.println(p2.equals(p1));
    }

} // end of paresIgualesMem
```

Redefinición de Métodos

Los lenguajes orientados a objetos poseen un mecanismo para "invocar" el método correcto en tiempo de ejecución, denominado dynamic dispatch.

```
public class paresIgualesMem {

    public static void main(String[] args) {

       SwappableIntPair p1 = new SwappableIntPair();
       MemoryIntPair p2 = new MemoryIntPair();
       p1.changeFst(1);
       p2.changeFst(2);
       System.out.println(p2.equals(p1));
    }

} // end of paresIgualesMem
```

invoca la versión original de changeFst

Redefinición de Métodos

Los lenguajes orientados a objetos poseen un mecanismo para "invocar" el método correcto en tiempo de ejecución, denominado dynamic dispatch.

invoca la versión nueva (de MemoryIntPair) de changeFst

```
public class paresIgualesMem {

    public static void main(String[] args) {

        SwappableIntPair p1 = new SwappableIntPair();
        MemoryIntPair p2 = new MemoryIntPair();
        p1.changeFst(1);
        p2.changeFst(2);
        System.out.println(p2.equals(p1));
    }

} // end of paresIgualesMem
```

invoca la versión original de changeFst

Herencia vs. Agregación

Herencia vs. Agregación

Además de usar herencia, uno puede usar agregación para representar, de alguna manera, una extensión de módulo.

Herencia vs. Agregación

Además de usar herencia, uno puede usar agregación para representar, de alguna manera, una extensión de módulo.

A modo de regla, debería usarse herencia para extender módulos sólo si el modulo que extiende corresponde a un subtipo del módulo extendido. Sino, debería utilizarse agregación.

Herencia vs. Agregación

Además de usar herencia, uno puede usar agregación para representar, de alguna manera, una extensión de módulo.

A modo de regla, debería usarse herencia para extender módulos sólo si el modulo que extiende corresponde a un subtipo del módulo extendido. Sino, debería utilizarse agregación.

```
public class SwappablePair {
    private Pair p;
    ...
    public void changeFst(int v1) {
        p.changeFst(v1);
    }
    ...
    public void swap() {
        int aux = p.fst();
        p.changeFst(p.snd());
        p.changeSnd(aux);
    }
}
```

Clases Abstractas

Clases Abstractas

Los lenguajes de programación orientados a objetos suelen permitir la definición de clases abstractas. Una clase abstracta es una clase con una implementación parcial.

Clases Abstractas

Los lenguajes de programación orientados a objetos suelen permitir la definición de clases abstractas. Una clase abstracta es una clase con una implementación parcial.

Una clase abstracta X no puede tener instancias concretas (pues no cuenta con una implementación completa). Las clases abstractas pueden utilizarse para definir interfaces, o servicios, que otras clases deberían poseer. Esto permite explotar el polimorfismo asociado a la herencia de clases.

Clases Abstractas

Los lenguajes de programación orientados a objetos suelen permitir la definición de clases abstractas. Una clase abstracta es una clase con una implementación parcial.

Una clase abstracta X no puede tener instancias concretas (pues no cuenta con una implementación completa). Las clases abstractas pueden utilizarse para definir interfaces, o servicios, que otras clases deberían poseer. Esto permite explotar el polimorfismo asociado a la herencia de clases.

En Java, las clases abstractas se declaran con la palabra reservada abstract. Java tambien soporta interfaces, que tienen una finalidad similar (pero están asociadas a la falta de herencia múltiple en en el lenguaje).

Clases Abstractas

Los lenguajes de programación orientados a objetos suelen permitir la definición de clases abstractas. Una clase abstracta es una clase con una implementación parcial.

Una clase abstracta X no puede tener instancias concretas (pues no cuenta con una implementación completa). Las clases abstractas pueden utilizarse para definir interfaces, o servicios, que otras clases deberían poseer. Esto permite explotar el polimorfismo asociado a la herencia de clases.

En Java, las clases abstractas se declaran con la palabra reservada abstract. Java tambien soporta interfaces, que tienen una finalidad similar (pero están asociadas a la falta de herencia múltiple en en el lenguaje).

Entre las clases predefinidas de Java, existe una clase especial denominada Object. Object es una clase abstracta, de la cual heredan implícitamente todas las clases.

Explotando Herencia para Definir Estructuras Polimórficas

Explotando Herencia para Definir Estructuras Polimórficas

La clase IntPair que definimos tiene el inconveniente que nos permite solamente representar pares ordenados de números enteros. Si necesitara, por ejemplo, pares ordenados de números reales, debería definirme otra clase.

Explotando Herencia para Definir Estructuras Polimórficas

La clase IntPair que definimos tiene el inconveniente que nos permite solamente representar pares ordenados de números enteros. Si necesitara, por ejemplo, pares ordenados de números reales, debería definirme otra clase.

Podemos explotar la herencia para definir pares ordenados que admitan cualquier tipo de objetos como componentes de cada par. Esto podemos lograrlo de la siguiente manera:

Explotando Herencia para Definir Estructuras Polimórficas

La clase IntPair que definimos tiene el inconveniente que nos permite solamente representar pares ordenados de números enteros. Si necesitara, por ejemplo, pares ordenados de números reales, debería definirme otra clase.

Podemos explotar la herencia para definir pares ordenados que admitan cualquier tipo de objetos como componentes de cada par. Esto podemos lograrlo de la siguiente manera:

```
public class Pair {
    private Object x;
    private Object y;

    public Pair() {
       x = null;
       y = null;
    }

    public Pair(Object v1, Object v2) {
       x = v1;
       y = v2;
    }

   public void changeFst(Object v1) {
       x = v1;
    }

    ...

    public Object fst() {
       return x;
    }

    ...

}
```

Uso de Estructuras Polimórficas

Uso de Estructuras Polimórficas

Ahora que contamos con pares polimórficos, estaríamos en condiciones de usarlos en lugar de los pares de enteros. Esperaríamos poder hacer, por ejemplo, algo como lo siguiente:

Uso de Estructuras Polimórficas

Ahora que contamos con pares polimórficos, estaríamos en condiciones de usarlos en lugar de los pares de enteros. Esperaríamos poder hacer, por ejemplo, algo como lo siguiente:

```
public static Pair minMaxArrayRevisited(int[] A, int i, int j) {

  if (i>j) {
      Pair res = new Pair(0,0);
      return res; // special case: we return (0,0) if the array is empty
  }
  else {
      Pair res_sub = new Pair(A[i],A[i]);
      int k = i;
      while (k<=j) {
          if (A[k]<res_sub.fst()) {
                res_sub.changeFst(A[k]);
            }
          if (A[k]>res_sub.snd()) {
                res_sub.changeSnd(A[k]);
            }
            k++;
      }
      return res_sub;
  }
} // end of minMaxArrayRevisited
```

Uso de Estructuras Polimórficas

Ahora que contamos con pares polimórficos, estaríamos en condiciones de usarlos en lugar de los pares de enteros. Esperaríamos poder hacer, por ejemplo, algo como lo siguiente:

```
public static Pair minMaxArrayRevisited(int[] A, int i, int j) {

  if (i>j) {
      Pair res = new Pair(0,0);
      return res; // special case: we return (0,0) if the array is empty
  }
  else {
      Pair res_sub = new Pair(A[i],A[i]);
      int k = i;
      while (k<=j) {
          if (A[k]<res_sub.fst()) {
              res_sub.changeFst(A[k]);
           }
          if (A[k]>res_sub.snd()) {
              res_sub.changeSnd(A[k]);
           }
           k++;
      }
      return res_sub;
  }
} // end of minMaxArrayRevisited
```

No puedo comparar instancias de Object con enteros!

Uso de Estructuras Polimórficas

Ahora que contamos con pares polimórficos, estaríamos en condiciones de usarlos en lugar de los pares de enteros. Esperaríamos poder hacer, por ejemplo, algo como lo siguiente:

```
public static Pair minMaxArrayRevisited(int[] A, int i, int j) {

  if (i>j) {
      Pair res = new Pair(0,0);
      return res; // special case: we return (0,0) if the array is empty
  }
  else {
      Pair res_sub = new Pair(A[i],A[i]);
      int k = i;
      while (k<=j) {
          if (A[k]<res_sub.fst()) {
              res_sub.changeFst(A[k]);
            }
          if (A[k]>res_sub.snd()) {
              res_sub.changeSnd(A[k]);
            }
            k++;
      }
      return res_sub;
  }
} // end of minMaxArrayRevisited
```

No puedo comparar instancias de Object con enteros!

Uso de Estructuras Polimórficas: Casts

Uso de Estructuras Polimórficas: Casts

Para poder usar estructuras polimórficas, debemos "transformar" las referencias, indicando que, si bien según el tipo de la estructura/método es X, por la manera en que lo usamos sabemos que los objetos almacenados en él son de tipo Y.

Uso de Estructuras Polimórficas: Casts

Para poder usar estructuras polimórficas, debemos "transformar" las referencias, indicando que, si bien según el tipo de la estructura/método es X, por la manera en que lo usamos sabemos que los objetos almacenados en él son de tipo Y.

Esto se realiza a través de casts. Dada cualquier expresión r que denote una referencia de tipo Clase1, puede convertirse en una referencia de tipo Clase2 mediante la siguiente expresión:

Uso de Estructuras Polimórficas: Casts

Para poder usar estructuras polimórficas, debemos "transformar" las referencias, indicando que, si bien según el tipo de la estructura/método es X, por la manera en que lo usamos sabemos que los objetos almacenados en él son de tipo Y.

Esto se realiza a través de casts. Dada cualquier expresión r que denote una referencia de tipo Clase1, puede convertirse en una referencia de tipo Clase2 mediante la siguiente expresión:

```
(Clase2) r
```

Uso de Estructuras Polimórficas: Casts

Para poder usar estructuras polimórficas, debemos "transformar" las referencias, indicando que, si bien según el tipo de la estructura/método es X, por la manera en que lo usamos sabemos que los objetos almacenados en él son de tipo Y.

Esto se realiza a través de casts. Dada cualquier expresión r que denote una referencia de tipo Clase1, puede convertirse en una referencia de tipo Clase2 mediante la siguiente expresión:

```
(Clase2) r
```

Si el objeto r no es de tipo Clase2, tendremos un error en tiempo de ejecución al querer usar alguna característica de r.

Uso de Estructuras Polimórficas: Casts

Para poder usar estructuras polimórficas, debemos "transformar" las referencias, indicando que, si bien según el tipo de la estructura/método es X, por la manera en que lo usamos sabemos que los objetos almacenados en él son de tipo Y.

Esto se realiza a través de casts. Dada cualquier expresión r que denote una referencia de tipo Clase1, puede convertirse en una referencia de tipo Clase2 mediante la siguiente expresión:

```
(Clase2) r
```

Si el objeto r no es de tipo Clase2, tendremos un error en tiempo de ejecución al querer usar alguna característica de r.

Los casts más seguros son los casts descendentes, es decir, casts en los cuales Clase2 es subclase de Clase1.

Uso de Casts

Uso de Casts

Volvamos al ejemplo anterior, y utilicemos los pares polimórficos de la manera correcta:

Uso de Casts

Volvamos al ejemplo anterior, y utilicemos los pares polimórficos de la manera correcta:

```
public static Pair minMaxArrayRevisited(int[] A, int i, int j) {

  if (i>j) {
      Pair res = new Pair(0,0);
      return res; // special case: we return (0,0) if the array is empty
  }
  else {
      Pair res_sub = new Pair(A[i],A[i]);
      int k = i;
      while (k<=j) {
          if (((Integer) res_sub.fst()).compareTo(A[k])>0) {
                res_sub.changeFst(A[k]);
            }
          if (((Integer) res_sub.snd()).compareTo(A[k])<0) {
                res_sub.changeSnd(A[k]);
            }
            k++;
      }
      return res_sub;
  }
} // end of minMaxArrayRevisited
```

Clases “Wrapper”

Clases “Wrapper”

En muchos casos necesitamos que las estructuras polimórficas que creemos admitan elementos de tipos básicos.

Clases “Wrapper”

En muchos casos necesitamos que las estructuras polimórficas que creemos admitan elementos de tipos básicos.

Dado que, para poder explotar el polimorfismo lo tenemos que hacer a través de clases, Java provee, por cada tipo básico, una clase “wrapper” correspondiente.

Clases “Wrapper”

En muchos casos necesitamos que las estructuras polimórficas que creemos admitan elementos de tipos básicos.

Dado que, para poder explotar el polimorfismo lo tenemos que hacer a través de clases, Java provee, por cada tipo básico, una clase “wrapper” correspondiente.

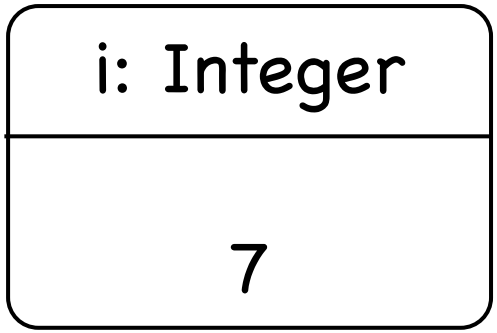
Ya vimos un ejemplo: Integer. Los objetos de tipo Integer contienen un valor de tipo int, y son inmutables.

Clases "Wrapper"

En muchos casos necesitamos que las estructuras polimórficas que creemos admitan elementos de tipos básicos.

Dado que, para poder explotar el polimorfismo lo tenemos que hacer a través de clases, Java provee, por cada tipo básico, una clase "wrapper" correspondiente.

Ya vimos un ejemplo: Integer. Los objetos de tipo Integer contienen un valor de tipo int, y son inmutables.

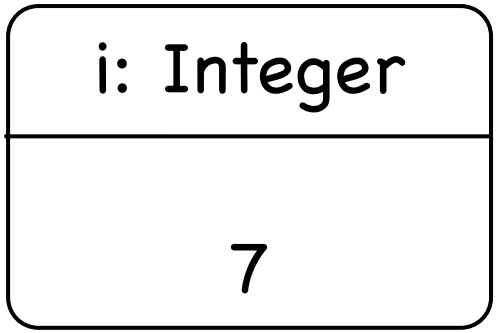


Clases "Wrapper"

En muchos casos necesitamos que las estructuras polimórficas que creemos admitan elementos de tipos básicos.

Dado que, para poder explotar el polimorfismo lo tenemos que hacer a través de clases, Java provee, por cada tipo básico, una clase "wrapper" correspondiente.

Ya vimos un ejemplo: Integer. Los objetos de tipo Integer contienen un valor de tipo int, y son inmutables.



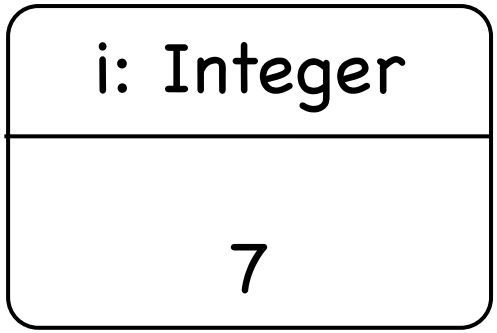
Para acceder al valor de i lo hacemos a través de i.intValue()

Clases "Wrapper"

En muchos casos necesitamos que las estructuras polimórficas que creemos admitan elementos de tipos básicos.

Dado que, para poder explotar el polimorfismo lo tenemos que hacer a través de clases, Java provee, por cada tipo básico, una clase "wrapper" correspondiente.

Ya vimos un ejemplo: Integer. Los objetos de tipo Integer contienen un valor de tipo int, y son inmutables.



Para acceder al valor de i lo hacemos a través de i.intValue()

A partir de su versión 1.5, Java ofrece muchas facilidades para "combinar" valores de tipos básicos y objetos de clases wrapper para el mismo tipo (vean por ejemplo el uso del constructor de Pair en el ejemplo anterior!)

Ejemplo: Resolviendo Problemas usando Programación Orientada a Objetos con Herencia (y Polimorfismo)

Ejemplo: Resolviendo Problemas usando Programación Orientada a Objetos con Herencia (y Polimorfismo)

Consideremos el siguiente problema:

Ejemplo: Resolviendo Problemas usando Programación Orientada a Objetos con Herencia (y Polimorfismo)

Consideremos el siguiente problema:

> Escriba un programa (en Java) que, dado un polinomio de grado dos, calcule las raíces del mismo.

Ejemplo: Resolviendo Problemas usando Programación Orientada a Objetos con Herencia (y Polimorfismo)

Consideremos el siguiente problema:

> Escriba un programa (en Java) que, dado un polinomio de grado dos, calcule las raíces del mismo.

Intentemos resolverlo entre todos...

Resumen de la Clase

Resumen de la Clase

Hemos visto una breve introducción a una característica particular de la programación orientada a objetos, denominada herencia. La herencia permite definir jerarquías de los tipos definidos por medio de clases, y reutilizar características de las clases involucradas en la jerarquía.

Resumen de la Clase

Hemos visto una breve introducción a una característica particular de la programación orientada a objetos, denominada herencia. La herencia permite definir jerarquías de los tipos definidos por medio de clases, y reutilizar características de las clases involucradas en la jerarquía.

Hemos visto que herencia corresponde simultáneamente a dos conceptos: extensión de módulos, y especialización de tipos.

Resumen de la Clase

Hemos visto una breve introducción a una característica particular de la programación orientada a objetos, denominada herencia. La herencia permite definir jerarquías de los tipos definidos por medio de clases, y reutilizar características de las clases involucradas en la jerarquía.

Hemos visto que herencia corresponde simultáneamente a dos conceptos: extensión de módulos, y especialización de tipos.

Un tipo particular de clases, las llamadas clases abstractas, corresponden a módulos parcialmente implementados, y pueden usarse para definir interfaces de otras clases. Una en particular, la clase Object, es abstracta y todas las clases en Java heredan de ésta implicitamente.

Resumen de la Clase

Hemos visto una breve introducción a una característica particular de la programación orientada a objetos, denominada herencia. La herencia permite definir jerarquías de los tipos definidos por medio de clases, y reutilizar características de las clases involucradas en la jerarquía.

Hemos visto que herencia corresponde simultáneamente a dos conceptos: extensión de módulos, y especialización de tipos.

Un tipo particular de clases, las llamadas clases abstractas, corresponden a módulos parcialmente implementados, y pueden usarse para definir interfaces de otras clases. Una en particular, la clase Object, es abstracta y todas las clases en Java heredan de ésta implicitamente.

Vimos además cómo podemos hacer para explotar herencia para conseguir definir estructuras polimórficas.